

Below is a structured lab guide with **sub-questions**, **Snort rules (custom messages)**, and **Kali Linux test commands** for each requirement.

Assumptions

- Web Server: 192.168.74.10
- Kali: 192.168.74.20
- HOME_NET = Web Server
- EXTERNAL_NET = any
- Local rules file: /etc/snort/rules/local.rules

1. Oversized ICMP Echo Request (>128 Bytes)

Objective

Detect ICMP Echo Requests larger than 128 bytes.

IDS Rule

```
alert icmp any any -> $HOME_NET any (  
  msg:"LAB-01 Oversized ICMP Echo Request (>128 Bytes)";  
  itype:8;  
  dsize:>128;  
  sid:1000001;  
  rev:1;  
)
```

IPS Rule

```
drop icmp any any -> $HOME_NET any (  
  msg:"IPS BLOCK: Oversized ICMP Packet";  
  itype:8;  
  dsize:>128;  
  sid:2000001;  
  rev:1;  
)
```

Kali Test

Linux ping: \$ping -s 200 192.168.1.10

Using Scapy: \$ send(IP(dst="192.168.1.10")/ICMP()/("A"*200))

2. ICMP Flood (30 packets in 1 minute)

IDS Rule

```
alert icmp any any -> $HOME_NET any ( msg:"LAB-02 ICMP Flood Detected"; itype:8; detection_filter:track by_src,count 30,seconds 60; sid:1000002; rev:1; )
```

IPS Rule

```
drop icmp any any -> $HOME_NET any ( msg:"IPS BLOCK: ICMP Flood"; itype:8; detection_filter:track by_src,count 30,seconds 60; sid:2000002; rev:1; )
```

Kali Test

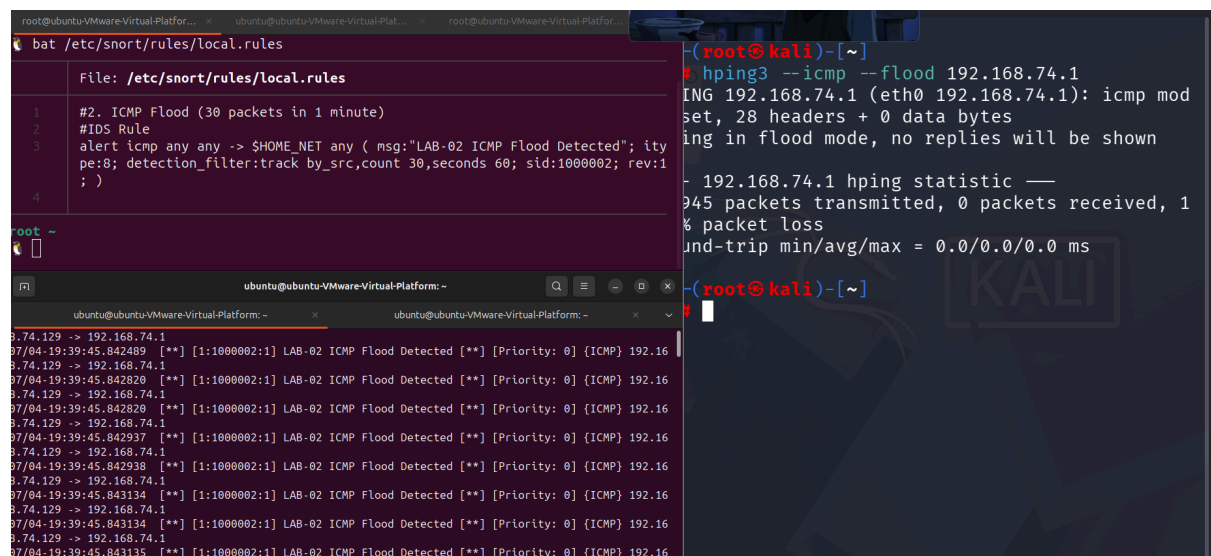
ping -f 192.168.1.10

or

hping3 --icmp --flood 192.168.1.10

Scapy

```
send(IP(dst="192.168.1.10")/ICMP(),count=40)
```



The screenshot displays a Kali Linux terminal environment. On the left, a file editor shows the configuration of an IDS rule in `/etc/snort/rules/local.rules`. The rule is an alert for ICMP flood attacks, configured with a detection filter that tracks by source IP, a count of 30 packets, and a time window of 60 seconds. The rule ID is 1000002. On the right, the terminal shows the execution of the `hping3` command to perform an ICMP flood attack on the IP address 192.168.74.1. The output indicates that 945 packets were transmitted, 0 were received, and there was 100% packet loss. The round-trip time statistics are also shown as 0.0/0.0/0.0 ms.

```
bat /etc/snort/rules/local.rules

File: /etc/snort/rules/local.rules

1 #2. ICMP Flood (30 packets in 1 minute)
2 #IDS Rule
3 alert icmp any any -> $HOME_NET any ( msg:"LAB-02 ICMP Flood Detected"; itype:8; detection_filter:track by_src,count 30,seconds 60; sid:1000002; rev:1; )
4

root ~

ubuntu@ubuntu-VMware-Virtual-Platform: ~
ubuntu@ubuntu-VMware-Virtual-Platform: ~
ubuntu@ubuntu-VMware-Virtual-Platform: ~

8.74.129 -> 192.168.74.1
87/04-19:39:45.842489 [**] [1:1000002:1] LAB-02 ICMP Flood Detected [**] [Priority: 0] {ICMP} 192.16
8.74.129 -> 192.168.74.1
87/04-19:39:45.842820 [**] [1:1000002:1] LAB-02 ICMP Flood Detected [**] [Priority: 0] {ICMP} 192.16
8.74.129 -> 192.168.74.1
87/04-19:39:45.842820 [**] [1:1000002:1] LAB-02 ICMP Flood Detected [**] [Priority: 0] {ICMP} 192.16
8.74.129 -> 192.168.74.1
87/04-19:39:45.842937 [**] [1:1000002:1] LAB-02 ICMP Flood Detected [**] [Priority: 0] {ICMP} 192.16
8.74.129 -> 192.168.74.1
87/04-19:39:45.842938 [**] [1:1000002:1] LAB-02 ICMP Flood Detected [**] [Priority: 0] {ICMP} 192.16
8.74.129 -> 192.168.74.1
87/04-19:39:45.843134 [**] [1:1000002:1] LAB-02 ICMP Flood Detected [**] [Priority: 0] {ICMP} 192.16
8.74.129 -> 192.168.74.1
87/04-19:39:45.843134 [**] [1:1000002:1] LAB-02 ICMP Flood Detected [**] [Priority: 0] {ICMP} 192.16
8.74.129 -> 192.168.74.1
87/04-19:39:45.843135 [**] [1:1000002:1] LAB-02 ICMP Flood Detected [**] [Priority: 0] {ICMP} 192.16

(root@kali)-[~]
# hping3 --icmp --flood 192.168.74.1
PING 192.168.74.1 (eth0 192.168.74.1): icmp mod
set, 28 headers + 0 data bytes
ing in flood mode, no replies will be shown

- 192.168.74.1 hping statistic ---
945 packets transmitted, 0 packets received, 1
% packet loss
und-trip min/avg/max = 0.0/0.0/0.0 ms

(root@kali)-[~]
```

3. TCP SYN Flood

IDS Rule

```
alert tcp any any -> $HOME_NET 80 ( msg:"LAB-03 TCP SYN Flood";  
flags:S; detection_filter:track by_src,count 20,seconds 60; sid:1000003;  
rev:1; )
```

IPS Rule

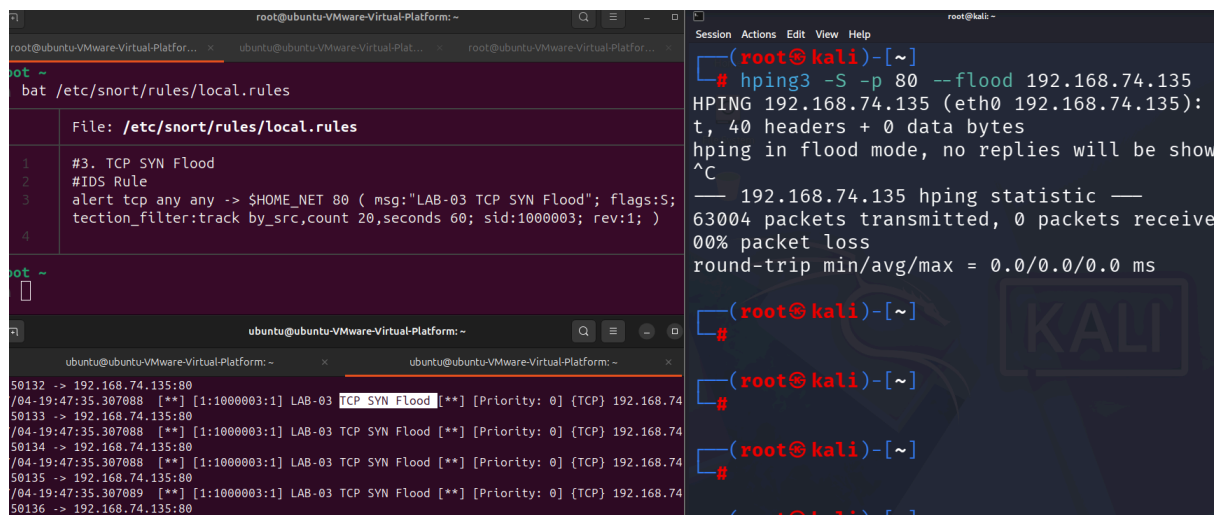
```
drop tcp any any -> $HOME_NET 80 (  
msg:"IPS BLOCK: TCP SYN Flood";  
flags:S;  
detection_filter:track by_src,count 20,seconds 60;  
sid:2000003;  
rev:1;  
)
```

Kali Test

```
hping3 -S -p 80 --flood 192.168.1.10
```

Scapy

```
send(IP(dst="192.168.1.10")/TCP(dport=80,flags="S"),count=30)
```



```
root@ubuntu-VMware-Virtual-Platform: ~  
root@ubuntu-VMware-Virtual-Platform: ~  
bat /etc/snort/rules/local.rules  
File: /etc/snort/rules/local.rules  
1 #3. TCP SYN Flood  
2 #IDS Rule  
3 alert tcp any any -> $HOME_NET 80 ( msg:"LAB-03 TCP SYN Flood"; flags:S;  
4 tecton_filter:track by_src,count 20,seconds 60; sid:1000003; rev:1; )  
root ~  
ubuntu@ubuntu-VMware-Virtual-Platform: ~  
50132 -> 192.168.74.135:80  
/04-19:47:35.307088 [**] [1:1000003:1] LAB-03 TCP SYN Flood [**] [Priority: 0] [TCP] 192.168.74  
50133 -> 192.168.74.135:80  
/04-19:47:35.307088 [**] [1:1000003:1] LAB-03 TCP SYN Flood [**] [Priority: 0] [TCP] 192.168.74  
50134 -> 192.168.74.135:80  
/04-19:47:35.307088 [**] [1:1000003:1] LAB-03 TCP SYN Flood [**] [Priority: 0] [TCP] 192.168.74  
50135 -> 192.168.74.135:80  
/04-19:47:35.307089 [**] [1:1000003:1] LAB-03 TCP SYN Flood [**] [Priority: 0] [TCP] 192.168.74  
50136 -> 192.168.74.135:80  
root@kali: ~  
root@kali: ~  
# hping3 -S -p 80 --flood 192.168.74.135  
HPING 192.168.74.135 (eth0 192.168.74.135):  
t, 40 headers + 0 data bytes  
hping in flood mode, no replies will be shown  
^C  
— 192.168.74.135 hping statistic —  
63004 packets transmitted, 0 packets received  
00% packet loss  
round-trip min/avg/max = 0.0/0.0/0.0 ms  
root@kali: ~  
root@kali: ~  
root@kali: ~  
root@kali: ~  
root@kali: ~
```

4. SSH Brute Force

IDS Rule

```
alert tcp any any -> $HOME_NET 22 ( \
  msg:"LAB-04 SSH Brute Force Attempt"; \
  detection_filter:track by_src, count 10, seconds 60; \
  sid:1000004; \
  rev:1;\
)
```

IPS Rule

```
drop tcp any any -> $HOME_NET 22 (
msg:"IPS BLOCK: SSH Brute Force";
flags:S;
detection_filter:track by_src,count 10,seconds 60;
sid:2000004;
rev:1;
)
```

Kali Test

```
hydra -l root -P rockyou.txt ssh://192.168.1.10
```

or

```
for i in {1..20}
```

do

```
ssh fake@192.168.1.10
```

done

The screenshot shows a Kali Linux terminal window with three panes. The top pane displays the contents of the file `/etc/snort/rules/local.rules`, which defines a rule named "LAB-04 SSH Brute Force Attempt". The middle pane shows the user's prompt as `root ~`. The bottom pane shows the command `sudo snort -q -A console -c /etc/snort/snort.conf` being executed, followed by several log entries indicating successful SSH brute force attempts from IP addresses like 192.168.74.129 and 192.168.74.135.

```
File: /etc/snort/rules/local.rules

#4. SSH Brute Force
#IDS Rule
alert tcp any any -> $HOME_NET 22 (\
msg:"LAB-04 SSH Brute Force Attempt"; \
detection filter: track by_src, count 10, seconds 60; \
sid:1000004; \
rev:1;\
)

root ~

ubuntu@ubuntu-Virtual-Platform: ~$ sudo snort -q -A console -c /etc/snort/snort.conf
07/04:20:15:41.884030 ** [1:1000004:1] LAB-04 SSH Brute Force Attempt ** [Priority: 0] {TCP} 192.168.74.129:35350 -> 192.168.74.135:22
07/04:20:15:41.886466 *** [1:1000004:1] LAB-04 SSH Brute Force Attempt *** [Priority: 0] {TCP} 192.168.74.129:35350 -> 192.168.74.135:22
07/04:20:15:41.886469 ** [1:1000004:1] LAB-04 SSH Brute Force Attempt ** [Priority: 0] {TCP} 192.168.74.129:35350 -> 192.168.74.135:22
07/04:20:15:41.891140 *** [1:1000004:1] LAB-04 SSH Brute Force Attempt *** [Priority: 0] {TCP} 192.168.74.129:35350 -> 192.168.74.135:22
07/04:20:15:42.107460 ** [1:1000004:1] LAB-04 SSH Brute Force Attempt ** [Priority: 0] {TCP} 192.168.74.129:35362 -> 192.168.74.135:22
07/04:20:15:43.108250 *** [1:1000004:1] LAB-04 SSH Brute Force Attempt *** [Priority: 0] {TCP} 192.168.74.129:35362 -> 192.168.74.135:22
```

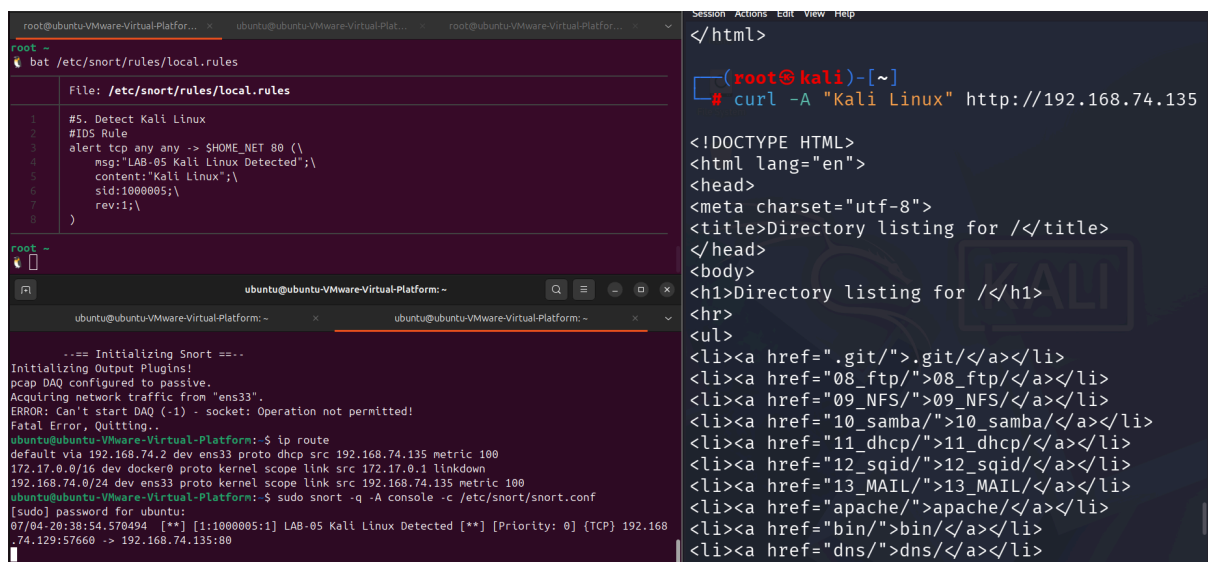
5. Detect Kali Linux

IDS Rule

```
alert http any any -> any any ( \
msg:"LAB-05 Possible Kali Linux User-Agent"; \
content:"Kali Linux"; \
sid:1000005; \
rev:1; \
)
```

Test

curl -A "Kali Linux" http://192.168.1.10



```
root@ubuntu-VMware-Virtual-Platform: ~
bat /etc/snort/rules/local.rules

File: /etc/snort/rules/local.rules
1 #5. Detect Kali Linux
2 #IDS Rule
3 alert tcp any any -> $HOME_NET 80 ( \
4   msg:"LAB-05 Kali Linux Detected"; \
5   content:"Kali Linux"; \
6   sid:1000005; \
7   rev:1; \
8 )

root@ubuntu-VMware-Virtual-Platform: ~
ubuntu@ubuntu-VMware-Virtual-Platform: ~
ubuntu@ubuntu-VMware-Virtual-Platform: ~

...== Initializing Snort ==...
Initializing Output Plugins!
pcap DAQ configured to passive.
Acquiring network traffic from "ens33".
ERROR: Can't start DAQ (-1) - socket: Operation not permitted!
Fatal Error, Quitting..
ubuntu@ubuntu-VMware-Virtual-Platform: ~$ ip route
default via 192.168.74.2 dev ens33 proto dhcp src 192.168.74.135 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.74.0/24 dev ens33 proto kernel scope link src 192.168.74.135 metric 100
ubuntu@ubuntu-VMware-Virtual-Platform: ~$ sudo snort -A console -c /etc/snort/snort.conf
[sudo] password for ubuntu:
07/04-20:38:54.578494 [**] [1:1000005:1] LAB-05 Kali Linux Detected [**] [Priority: 0] [TCP] 192.168
74.129:57660 -> 192.168.74.135:80

</html>
(root@kali)-[~]
# curl -A "Kali Linux" http://192.168.74.135

<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Directory listing for /</title>
</head>
<body>
<h1>Directory listing for /</h1>
<hr>
<ul>
<li><a href=".git/">.git/</a></li>
<li><a href="08_ftp/">08_ftp/</a></li>
<li><a href="09_NFS/">09_NFS/</a></li>
<li><a href="10_samba/">10_samba/</a></li>
<li><a href="11_dhcp/">11_dhcp/</a></li>
<li><a href="12_sqid/">12_sqid/</a></li>
<li><a href="13_MAIL/">13_MAIL/</a></li>
<li><a href="apache/">apache/</a></li>
<li><a href="bin/">bin/</a></li>
<li><a href="dns/">dns/</a></li>
```

6. Gmail Access

IDS Rule

```
alert udp any any -> any 53 (\
  msg:"LAB-06 Gmail DNS Query";\
  content:"gmail";\
  nocase;\
  sid:1000006;\
  rev:1;\
)
```

Test

firefox <https://gmail.com>

or

curl <https://gmail.com>

The screenshot displays a Kali Linux terminal environment with three windows. The top window shows the configuration of a Snort rule in `/etc/snort/rules/local.rules`. The rule is designed to detect DNS queries for 'gmail' on port 53. The middle window shows the execution of the Snort engine with the command `snort -q -i ens33 -A console -c /etc/snort/snort.conf`, which outputs several alert messages for 'LAB-06 Gmail DNS Query'. The bottom window shows the execution of a curl command to access <https://gmail.com>, which returns a 301 Moved status, and an IP command showing the network configuration of the interface `eth0`.

```
root@ubuntu-VMware-Virtual-Platform: ~
bat /etc/snort/rules/local.rules

File: /etc/snort/rules/local.rules
1 #6. Gmail Access
2 #IDS Rule
3 alert udp any any -> any 53 (\
4   msg:"LAB-06 Gmail DNS Query";\
5   content:"gmail";\
6   nocase;\
7   sid:1000006;\
8   rev:1;\
9 )

root ~
ubuntu@ubuntu-VMware-Virtual-Platform: ~
snort -q -i ens33 -A console -c /etc/snort/snort.conf
07/04-20:52:38.429261 00000006:1] LAB-06 Gmail DNS Query [0] [Priority: 0] {UDP} 192.168.74.129:35112 -> 192.168.74.2:53
07/04-20:52:38.429266 00000006:1] LAB-06 Gmail DNS Query [0] [Priority: 0] {UDP} 192.168.74.129:40540 -> 192.168.74.2:53
07/04-20:52:48.771020 00000006:1] LAB-06 Gmail DNS Query [0] [Priority: 0] {UDP} 192.168.74.129:55940 -> 192.168.74.2:53
07/04-20:52:48.771026 00000006:1] LAB-06 Gmail DNS Query [0] [Priority: 0] {UDP} 192.168.74.129:37166 -> 192.168.74.2:53

root@kali: ~
# curl https://gmail.com
<HTML><HEAD><meta http-equiv="content-type" content="text/html; charset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="https://mail.google.com/mail/u/0/">here</A>.
</BODY></HTML>

root@kali: ~
# ip -br a
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.74.129/24 fe80::1b6d:c5b0:2cd4:70ba/64
```

7. Facebook Access

```
alert udp any any -> any 53 (\
  msg:"LAB-06 Gmail DNS Query";\
  content:"Facebook";\
  nocase;\
  sid:1000006;\
  rev:1;\
)
```

Test

curl https://facebook.com

8. Instagram Access

```
alert udp any any -> any 53 (\
  msg:"LAB-06 Gmail DNS Query";\
  content:"instagram";\
  nocase;\
  sid:1000006;\
  rev:1;\
)
```

Test

curl https://instagram.com


```
root@ubuntu-Virtual-Platform:~# cat /etc/snort/rules/local.rules
#9. Port Scan
#IDS Rule
alert tcp any any -> $HOME_NET any (\
msg:"LAB-09 Port Scan";\
flags:S;\
threshold:type both,track by_src,count 20,seconds 10;\
sid:1000009;\
rev:1;\
)

root ~
[+] ubuntu@ubuntu-Virtual-Platform:~
ubuntu@ubuntu-Virtual-Platform:~$ sudo snort -q -A console -c /etc/snort/snort.conf
[sudo] password for ubuntu:
07/04-20:58:31.278470  [**] [1:1000009:1] LAB-09 Port Scan [**] [Priority: 0] [TCP] 192.168.74.129:38
279 -> 192.168.74.135:22

Session Actions Edit View Help
(root@kali)-[~]
# nmap -Pn 192.168.74.135
Starting Nmap 7.99SVN ( https://nmap.org ) at
2026-07-04 11:28 -0400
Nmap scan report for 192.168.74.135
Host is up (0.00055s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
MAC Address: 00:0C:29:DA:02:68 (VMware)

Nmap done: 1 IP address (1 host up) scanned in
0.76 seconds

(root@kali)-[~]
#
```

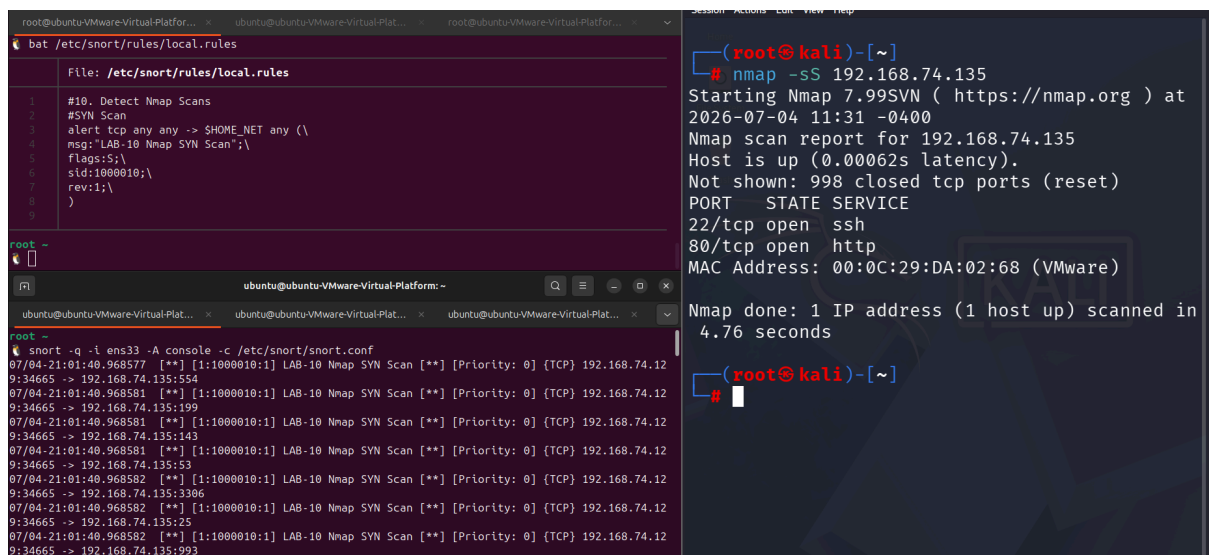
10. Detect Nmap Scans

SYN Scan

```
nmap -sS 192.168.1.10
```

Rule

```
alert tcp any any -> $HOME_NET any (\
msg:"LAB-10 Nmap SYN Scan";\
flags:S;\
sid:1000010;\
rev:1;\
)
```



The screenshot displays a Kali Linux terminal environment. On the left, a file editor shows the configuration of a Snort rule in `/etc/snort/rules/local.rules`. The rule is designed to detect Nmap SYN scans by alerting on TCP traffic with the SYN flag set. The rule ID is 1000010 and it is version 1. On the right, the terminal shows the execution of `nmap -sS 192.168.74.135`. The output indicates that the host is up, with a latency of 0.00062s, and that 998 closed TCP ports were reset. The scan also identified open ports 22 (SSH) and 80 (HTTP). The MAC address of the host is 00:0C:29:DA:02:68 (VMware). The scan completed in 4.76 seconds, scanning 1 IP address.

```
(root@kali)-[~]
# nmap -sS 192.168.74.135
Starting Nmap 7.99SVN ( https://nmap.org ) at
2026-07-04 11:31 -0400
Nmap scan report for 192.168.74.135
Host is up (0.00062s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
MAC Address: 00:0C:29:DA:02:68 (VMware)

Nmap done: 1 IP address (1 host up) scanned in
4.76 seconds

(root@kali)-[~]
#
```

FIN Scan

```
nmap -sF 192.168.1.10
```

Rule

```
alert tcp any any -> $HOME_NET any (
msg:"LAB-10 Nmap FIN Scan";
flags:F;
sid:1000011;
rev:1;
)
```

NULL Scan

```
nmap -sN 192.168.1.10
alert tcp any any -> $HOME_NET any (
msg:"LAB-10 Nmap NULL Scan";
flags:0;
sid:1000012;
rev:1;
)
```

XMAS Scan

```
nmap -sX 192.168.1.10
alert tcp any any -> $HOME_NET any (\
msg:"LAB-10 Nmap XMAS Scan";\
flags:FPU;\
sid:1000013;\
rev:1;\
)
```

The screenshot shows a Kali Linux terminal with three windows. The top window displays the configuration of Snort rules for NULL and XMAS scans. The middle window shows the execution of an nmap -sX scan on 192.168.74.135. The bottom window shows the output of the scan, indicating that the host is up and that 998 closed TCP ports were reset.

```
File: /etc/snort/rules/local.rules
1 #XMAS Scan
2 #nmap -sX 192.168.1.10
3 #Rule
4 alert tcp any any -> $HOME_NET any (
5 msg:"LAB-10 Nmap XMAS Scan";\
6 flags:FPU;\
7 sid:1000013;\
8 rev:1;\
9 )
10

root@kali:~# nmap -sX 192.168.74.135
Starting Nmap 7.99SVN ( https://nmap.org ) at
2026-07-04 11:33 -0400
Nmap scan report for 192.168.74.135
Host is up (0.00038s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE      SERVICE
22/tcp    open|filtered ssh
80/tcp    open|filtered http
MAC Address: 00:0C:29:DA:02:68 (VMware)

Nmap done: 1 IP address (1 host up) scanned in
1.90 seconds

root@kali:~#
```

Version Detection

```
nmap -sV 192.168.1.10
```

OS Detection

```
nmap -O 192.168.1.10
```

11. SQL Injection

IDS Rule

```
alert tcp any any -> $HOME_NET 80 (\
msg:"LAB-11 SQL Injection";\
content:"' OR 1=1";\
nocase;\
http_uri;\
sid:1000014;\
rev:1;\
)
```

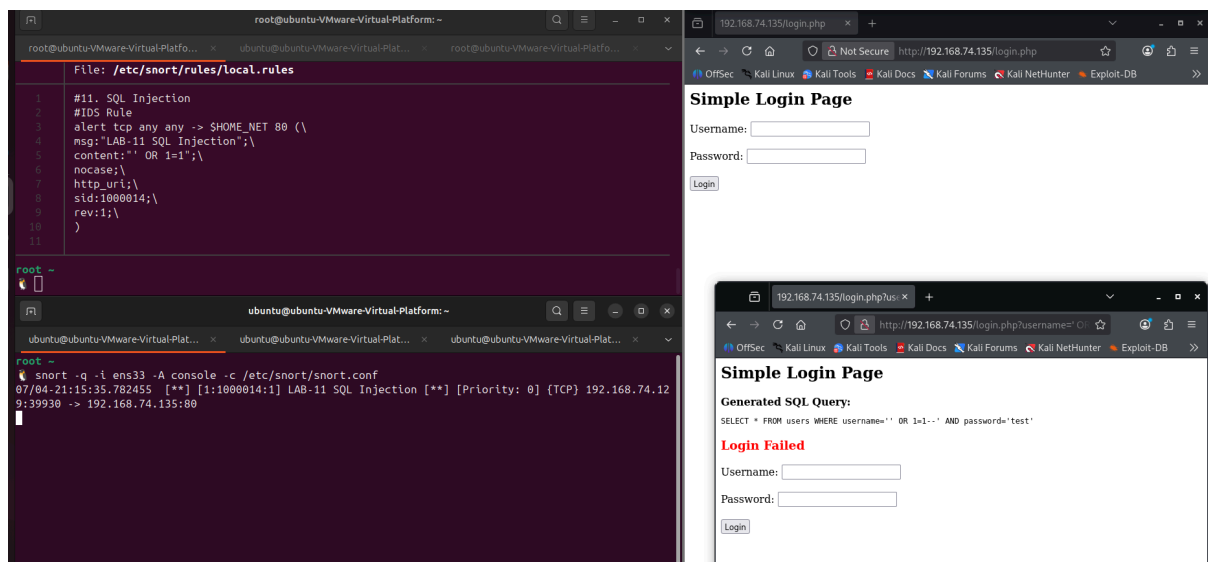
Kali Test

Browser

<http://192.168.1.10/login.php?user=admin' OR 1=1-->

or

[sqlmap -u "http://192.168.1.10/login.php?id=1"](http://192.168.1.10/login.php?id=1)



1. Install Apache + PHP

```
sudo apt update
sudo apt install apache2 php -y
```

2. Create the login page

```
sudo nano /var/www/html/login.php
```

Paste:

```
<?php
$username = $_GET['username'] ?? '';
$password = $_GET['password'] ?? '';

echo "<h2>Simple Login Page</h2>";

if ($username != "" || $password != "") {

    $query = "SELECT * FROM users WHERE username='$username' AND password='$password'";

    echo "<b>Generated SQL Query:</b><br>";
    echo "<pre>$query</pre>";

    if ($username == "admin" && $password == "admin123") {
        echo "<h3 style='color:green;'>Login Successful</h3>";
    } else {
        echo "<h3 style='color:red;'>Login Failed</h3>";
    }
}
?>

<form method="GET">
Username:
<input type="text" name="username"><br><br>

Password:
<input type="password" name="password"><br><br>

<input type="submit" value="Login">
</form>
```

3. Restart Apache

```
sudo systemctl restart apache2
```

4. Open from Kali

```
http://192.168.74.135/login.php
```

5. Test SQL Injection

Normal request

```
http://192.168.74.135/login.php?username=admin&password=admin123
```

SQL Injection

```
http://192.168.74.135/login.php?username=' OR 1=1--&password=test
```

or

```
curl "http://192.168.74.135/login.php?username=' OR 1=1--&password=test"
```

12. Cross Site Scripting

IDS Rule

```
alert http any any -> $HOME_NET 80 (  
  msg:"LAB-12 XSS Attack";  
  content:"<script>";  
  nocase;  
  http_uri;  
  sid:1000015;  
  rev:1;  
)
```

Kali Test

```
http://192.168.1.10/search.php?q=<script>alert(1)</script>
```

or

```
curl "http://192.168.1.10/?q=<script>alert(1)</script>"
```

13. Directory Traversal

IDS Rule

```
alert http any any -> $HOME_NET 80 (  
  msg:"LAB-13 Directory Traversal";  
  content:"../";  
  http_uri;  
  sid:1000016;  
  rev:1;  
)
```

Kali Test

```
http://192.168.1.10/../../../../etc/passwd
```

or

```
curl "http://192.168.1.10/../../../../etc/passwd"
```

Encoded test

```
curl "http://192.168.1.10/%2e%2e/%2e%2e/etc/passwd"
```

Recommended Lab Flow

For each exercise, document:

1. **Objective**
2. **Attack Description**
3. **Snort Rule**
4. **Rule Explanation**

Restart Snort

```
sudo systemctl restart snort
```

- 5.
6. **Run the Kali attack**

Capture the alert

```
sudo tail -f /var/log/snort/alert
```

- 7.
8. **Take screenshots of:**
 - Attack command on Kali
 - Snort alert
 - Wireshark packet (optional)
9. **Result**
10. **Conclusion**

This structure is suitable for a practical Snort IDS/IPS lab report and covers both the detection (IDS) and prevention (IPS) objectives in your assignment.

Here's what each of those Snort rule options means, with simple examples.

Attribute	Purpose	Example
msg	Custom alert message displayed when the rule matches.	<code>msg:"ICMP Flood Detected";</code>
sid	Signature ID. A unique identifier for the rule. Custom rules usually use IDs above <code>1000000</code> .	<code>sid:1000001;</code>
rev	Rule revision number. Increment it whenever you modify the rule.	<code>rev:2;</code>
itype	Matches a specific ICMP Type .	<code>itype:8;</code> → ICMP Echo Request (Ping)
dsize	Matches the packet payload size.	<code>dsize:>128;</code> → Payload larger than 128 bytes
detection_filter:track by_src,count 30,seconds 60;	Detects traffic rate. Generates an alert when one source IP sends 30 or more packets within 60 seconds .	Used for ICMP floods, brute-force, SYN floods
flags:F	Match TCP packets with only the FIN flag set.	<code>flags:F;</code>

	Often indicates a FIN scan.	
flags:P	Match packets with the PSH (Push) flag.	<code>flags:P;</code>
flags:U	Match packets with the URG (Urgent) flag.	<code>flags:U;</code>
flags:FPU	Match packets with FIN + PSH + URG flags (XMAS scan).	<code>flags:FPU;</code>
http_header	Restricts the search to the HTTP headers only.	<code>content:"Kali"; http_header;</code>
tls.sni	Searches the Server Name Indication (SNI) in a TLS (HTTPS) ClientHello before encryption.	<code>content:"gmail.com"; tls.sni;</code>
content	Searches for a specific text or byte sequence in the packet.	<code>content:"facebook.com";</code>
threshold:type both,track by_src,count 20,seconds 10;	Limits alerts. If a source sends 20 matching packets in 10 seconds , an alert is generated while suppressing	Port scan detection

excessive duplicates.
Useful for scans.

content:" OR 1=1";	Looks for a common SQL Injection payload in HTTP traffic.	Detects SQLi attempts
nocase	Makes the preceding content match case-insensitively .	Matches ADMIN , Admin , and admin
content:"";	Detects a common Cross-Site Scripting (XSS) payload.	<code><script>alert(1)</script></code>
content:"../";	Detects a Directory Traversal attempt.	<code>../../../../etc/passwd</code>
http_uri	Restricts the content search to the HTTP request URI (URL path/query) .	Detects <code>../</code> only in the requested URL

Difference: **detection_filter** vs **threshold**

- **detection_filter**: Detects **high-rate attacks** (e.g., flood, brute force). It alerts **after** the specified number of packets within the time window.
- **threshold**: Controls **how often alerts are generated** to avoid log flooding when many matching packets occur.

Common TCP Flags

Flag	Meaning	Used By
------	---------	---------

S	SYN	SYN Scan, connection initiation
A	ACK	Established connections
F	FIN	FIN Scan
R	RST	Connection reset
P	PSH	Push data immediately
U	URG	Urgent data
FPU	FIN + PSH + URG	XMAS Scan
0	No flags	NULL Scan

These are the attributes you'll use most often for your Snort IDS/IPS lab.